



Setup Guide

Local installation, credentials, and running the agent

Everything needed to run Health Sentinel end to end: environment setup, model/cost configuration, the Streamlit UI, and the eval suite that gates every change.

Python 3.12+

uv

OpenAI required

No other services needed

~10 min

PREREQUISITES



Python 3.12+

A working interpreter; the project ships a pyproject.toml managed by uv.



uv

Handles the virtual environment and locked dependency installs via uv sync.



OpenAI API key

Required — every pipeline stage (vision, nutrition, RAG, prediction, recommendation, critic) calls this model.



Nothing else required

All MCP connectors (iWatch, SMS, calendar, Google Drive) are simulated with deterministic synthetic data — no OAuth setup needed.

1. CLONE & INSTALL

```
# 1. Enter the project
cd healthsentinel

# 2. Install dependencies (creates .venv)
uv sync

# 3. Create your local environment file
cp .env.example .env
```

2. CONFIGURE CREDENTIALS

Copy `.env.example` to `.env` and fill in the values below. Only `OPENAI_API_KEY` is required — everything else has a sane default.

VARIABLE	PURPOSE	EXAMPLE
OPENAI_API_KEY	Required — powers every agent across all model tiers	sk-proj-...
HEALTHSENTINEL_CHEAP_MODEL	Vision + lab-report-image extraction tier	gpt-4o-mini-2024-07-18
HEALTHSENTINEL_MID_MODEL	Nutrition mapping, medical RAG, recommendations	gpt-4o-mini-2024-07-18
HEALTHSENTINEL_REASONING_MODEL	Prediction engine + advisory critic	gpt-4o-2024-08-06
HEALTHSENTINEL_MAX_RUN_COST_USD	Hard per-run spend cap before a degraded finalize	0.50
HEALTHSENTINEL_CHROMA_DIR	Medical-document vector index location	./data/chroma
HEALTHSENTINEL_METRICS_DB	SQLite historic metrics (glucose/weight/sleep)	./data/metrics.db

3. RUN THE APP

```
# From the project root
uv run streamlit run app.py

# Open in a browser
http://localhost:8501
```

4. RUN THE EVAL SUITE

```
# Deterministic checks only – no API key, no network, gates CI
uv run python eval/run_eval.py --offline

# Full suite – RAG + graph + adversarial red-team cases
uv run python eval/run_eval.py
```

5. AGENTS AT A GLANCE

AGENT / NODE	TIER	ROLE
vision_agent	cheap	Meal-photo food recognition; hallucination floor blocks low-confidence reads
nutrition_agent	mid (mapping only)	LLM maps food to canonical keys; totals are summed deterministically
medical_rag_agent	mid	RAG over data/medical_docs/*.md; refuses on weak evidence
activity_agent	—	iWatch MCP (simulated); persists sleep history
sms_agent	—	Ingests user-confirmed SMS signals (meal timing / gym / health)
calendar_agent	—	Google Calendar MCP (simulated); stress signal
lab_report_agent	cheap/mid	Extracts glucose/weight from an uploaded lab report; validates before persisting
trend_agent	none — deterministic	Classifies 90-day glucose/weight/sleep trends in pure Python
prediction_agent	reasoning	SAFE-category predictions + risk dashboard; category allowlist enforced
recommendation_agent	mid	Filtered, cited recommendations; allergen/bounds guardrails applied
critic_agent	reasoning — advisory	Second opinion; never overrides the deterministic verifier

TROUBLESHOOTING



OPENAI_API_KEY missing

The sidebar shows a not-found status and blocks the run; add it in the sidebar text box or `.env`.



RAG index looks empty

Click “(Re)build medical RAG index” in the sidebar to re-chunk and re-embed `data/medical_docs/*.md`.



A run is marked “degraded”

Expected when an agent recovered from a fault (e.g. a refused RAG lookup); expand “Agent errors this run” to see the typed fault class.



Port already in use

Free it with `lsof -ti tcp:8501 | xargs kill -9`, then re-run `streamlit run app.py`.

Ready to run in about 10 minutes.

No infrastructure to stand up — SQLite, Chroma, and the in-memory checkpointer are all local; only your API key is external.

- uv sync
- Configure .env
- streamlit run
- Log a meal